REPORT

Zajęcia: Analog and digital electronic circuits
Teacher: prof. dr hab. Vasyl Martsenyuk

Lab 6
Date 27.03.26
Topic: "Random Signal Analysis. Power Spectrum Estimation"
Variant 13

Kostiantyn Hrekov
Informatyka II stopień,
stacjonarne,
1 semestr,
Gr 1.a

1. Problem statement:

Variant 13.

Simulate two random signals with different power levels. Analyze and compare their energy and standard deviation. Estimate and compare their PSDs using Welch with Hamming and Hann windows.

Input data:

N = 5000 # number of samples
fs = 1000 # sampling frequency (Hz)
signal1 = np.random.normal(0, 1, N) # lower power
signal2 = np.random.normal(0, 3, N) # higher power

2. Commands used (or GUI):

```
import numpy as np
import matplotlib.pyplot as plt
from statsmodels.regression.linear_model import yule_walker
from spectrum import pburg

# ========================
# 1. Parameters
# ========================
np.random.seed(0)

N_short = 100
N_long = 5000

# Stable AR(2) coefficients
a1 = -0.75
a2 = 0.5


# ========================
# 2. Generate AR(2) signal
# ========================
def generate_ar2(N, a1, a2):
    noise = np.random.normal(0, 1, N)
    x = np.zeros(N)

    # initialize first samples
    x[0] = noise[0]
    x[1] = noise[1]

    for n in range(2, N):
        x[n] = -a1 * x[n - 1] - a2 * x[n - 2] + noise[n]

    return x


signal_short = generate_ar2(N_short, a1, a2)
signal_long = generate_ar2(N_long, a1, a2)

# ========================
# 3. Plot signals
# ========================
plt.figure(figsize=(10, 4))

plt.plot(signal_short, label="Short signal")
```

```
plt.plot(signal_long[:200], label="Long signal (first 200 samples)")

plt.title("AR(2) Signals")
plt.xlabel("Samples")
plt.ylabel("Amplitude")
plt.legend()
plt.grid()

plt.show()

# ========================
# 4. Yule-Walker estimation
# ========================
rho_short, sigma_short = yule_walker(signal_short, order=2)
rho_long, sigma_long = yule_walker(signal_long, order=2)

print("=== Yule-Walker ===")
print("Short signal AR coeffs:", -rho_short)
print("Long signal AR coeffs:", -rho_long)

# ========================
# 5. Burg estimation
# ========================
burg_short = pburg(signal_short, order=2, NFFT=512)
burg_long = pburg(signal_long, order=2, NFFT=512)

# Run Burg computation
burg_short()
burg_long()

print("\n=== Burg ===")
print("Short signal AR coeffs:", burg_short.ar)
print("Long signal AR coeffs:", burg_long.ar)

# ========================
# 6. PSD comparison (Burg)
# ========================
plt.figure(figsize=(10, 6))

burg_short.plot(label="Short signal (Burg)")
burg_long.plot(label="Long signal (Burg)")

plt.title("PSD using Burg method")
plt.grid()

plt.show()
```

Output:

=== Yule-Walker ===

Short signal AR coeffs: [-0.84869822 0.51324137]

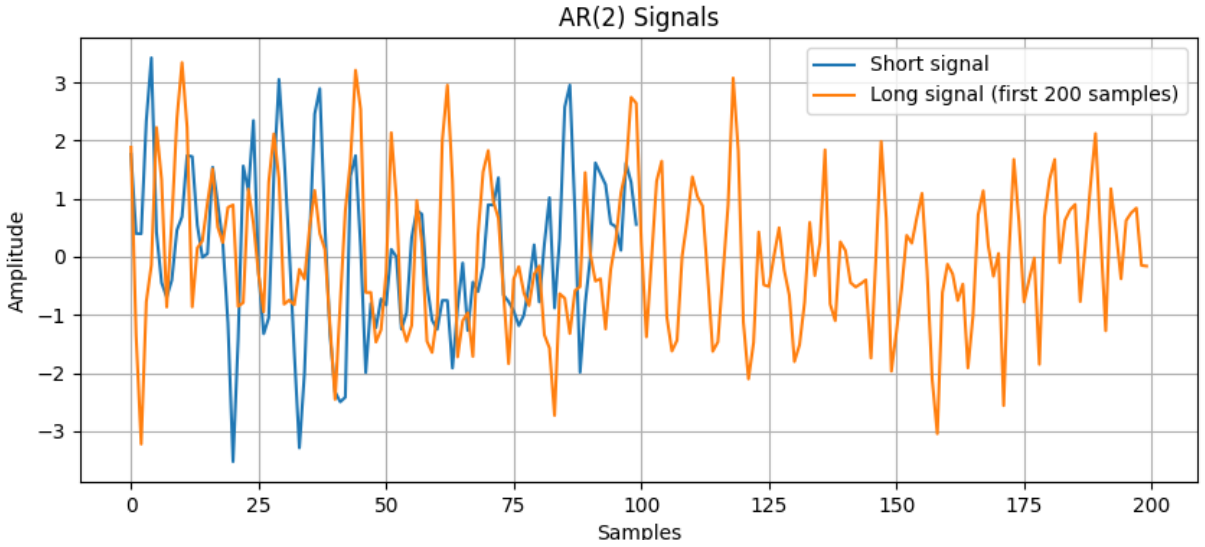
Long signal AR coeffs: [-0.76258998 0.50798338]

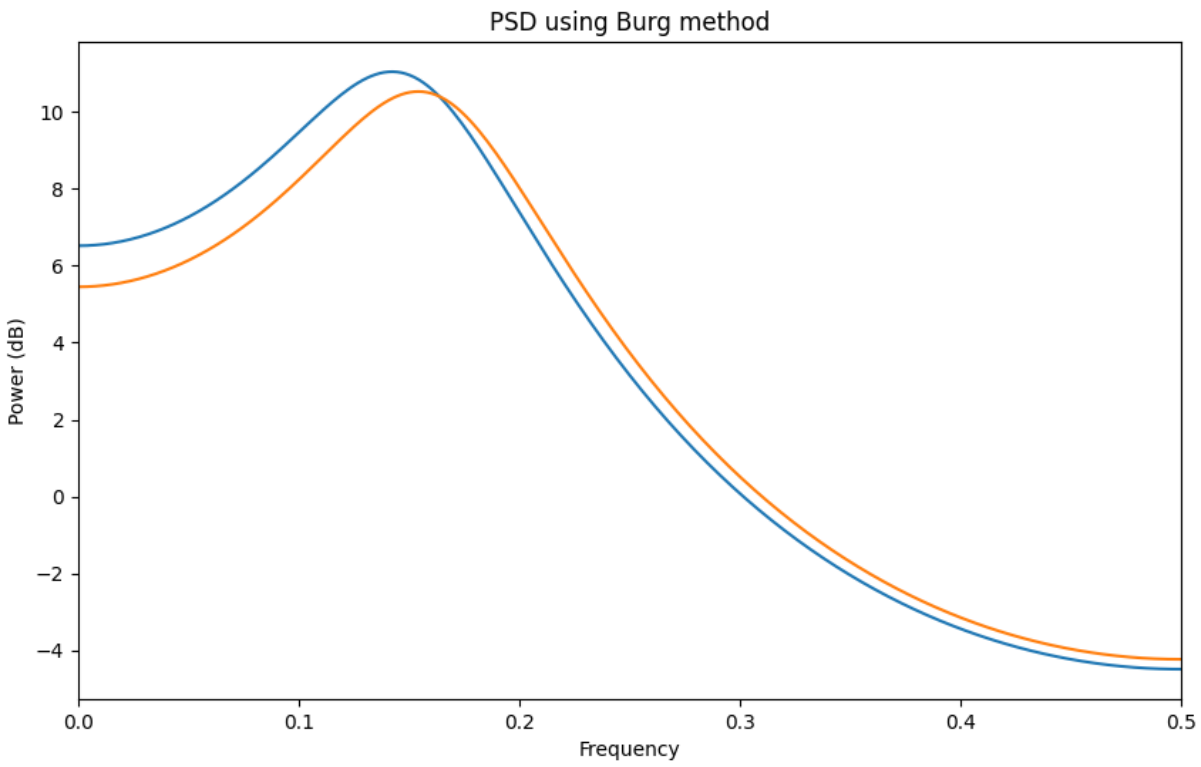
=== Burg ===

Short signal AR coeffs: [-0.84652721+0.j 0.51046055-0.j]

Long signal AR coeffs: [-0.76308981+0.j 0.50829321-0.j]

The results show that both the Burg and Yule-Walker methods produce very similar AR(2) coefficient estimates, with more accurate values obtained for the longer signal. This indicates that signal length has a greater impact on estimation accuracy than the choice of method.





Conclusion:

Due to this laboratory, we learned how multidimensional signal processing techniques can be applied to both image and audio data using tools such as the 2D Discrete Fourier Transform and spectrogram analysis. We observed how frequency-domain representations reveal important structural and temporal features, and how methods like STFT enable analysis of non-stationary signals by capturing their time–frequency behavior.

Link to remote repozytorium (e.g. GitHub):

https://github.com/KostiaGrekov/DSPja/tree/main/DSPja%206